

```
"""
Q-Learning for Mastermind
=====
```

YOUR TASKS:

1. Complete the `select_action` and `update` methods in the provided `QLearningAgent` class
2. Find the optimal history length for the state representation
3. Investigate a good exploration rate or regime (`epsilon`)
4. Plot learning curves and comparison charts with proper statistics
5. Write a report analyzing your results and answering the questions at the end of this file.

For the experiments (tasks 2-4), the overall experiment structure is already provided below. Once your implementation of `select_action` and `update` is working, you should be able to run these functions directly and then adapt the settings and report your findings.

CONFIGURATION:

It is recommended to use `NUM_COLORS = 4` by default. If the code runs too slowly on your machine, you may reduce `NUM_COLORS` from 4 to 3, but please state this clearly in your report and use the same configuration for the random baseline comparisons.

In many reinforcement learning studies, one would also tune the learning rate (`eta`) and the discount factor (`gamma`). To keep this study short and focused, use the reasonable fixed values provided in the code (`eta = 0.2`, `gamma = 0.99`) while investigating `history_length` and `epsilon`.

For experiments, it is recommended to use enough independent runs for your qualitative conclusions to look steady. You may find it helpful to start with tens of runs rather than hundreds, but increase the number if your results are still unstable.

A sensible order is to investigate `history_length` first, then investigate `epsilon` using your chosen history setting, and finally plot learning curves for your final combined choice.

In your report, clearly state the parameter values you used for each experiment. This should include the tested `history_lengths`, the tested `epsilon` values or range, the final chosen `history_length` and `epsilon`, and the main run/evaluation settings used to produce your reported tables and plots.

PROVIDED COMPONENTS:

- `MastermindEnv` is provided in `environment.py`
- `QTable` is provided in `qtable.py`

ENVIRONMENT API YOU NEED:

- `env = MastermindEnv(...)` creates the environment
- `state = env.reset()` starts a new episode and returns the initial state
- `next_state, reward, done, info = env.step(action)` takes one action
- `env.num_codes` is the number of possible actions
- `env.done` tells you whether the current episode has finished
- `env.turn` is the current turn number

STATE FORMAT:

- The state is a tuple of the last `history_length` turn records
- Each turn record is `(guess_pos1, guess_pos2, black, white)`
- Empty slots are `None`

STEP OUTPUTS:

- `next_state`: the next state tuple
- `reward`: `10.0` if the current guess solves the code, otherwise `-1.0`
- `done`: `True` if the episode has ended, otherwise `False`
- `info`: dictionary containing `guess`, `black`, `white`, `turn`, and `won`

REWARD SCHEME:

- positive reward: `10.0` only when the agent wins
- negative reward: `-1.0` on every non-winning step
- this includes the final step of a lost game

Q-TABLE API YOU NEED:

- `Q.get(state, action)` returns `Q(state, action)`
- `Q.set(state, action, value)` stores `Q(state, action) = value`
- `Q.get_max_value(state)` returns `max_a Q(state, a)`
- `Q.get_best_action(state)` returns a greedy action, breaking ties randomly
- unseen state-action values default to `0.0`

This means that, especially early in training, many actions may be tied and a greedy choice can still move randomly among those tied actions.

Please use the provided `QTable` as given in this assignment. You should not need to modify `environment.py` or `qtable.py`.

```
"""

from typing import Tuple
```

```
import matplotlib.pyplot as plt
import numpy as np

from environment import MastermindEnv
from qtable import QTable

# =====
# CONFIGURATION - adjust if needed
# =====

NUM_COLORS = 4
NUM_POSITIONS = 2
MAX_TURNS = 6

# Fixed RL parameters for this assignment
ETA = 0.2 # Learning rate held fixed in this assignment
GAMMA = 0.99 # Discount factor held fixed in this assignment

# Smoke-test settings
SMOKE_TEST_HISTORY_LENGTH = 2 # Debugging-only setting; not a recommendation for the experiments
SMOKE_TEST_EPSILON = 0.15 # Exploration rate used in the implementation check
SMOKE_TEST_NUM_EPISODES = 10_000 # Training episodes used in the implementation check
SMOKE_TEST_EVAL_INTERVAL = 2_000 # Training episodes between two progress measurements in the implementation check
SMOKE_TEST_CURVE_EVAL_GAMES = 200 # Evaluation games averaged into each progress point during the implementation check
SMOKE_TEST_FINAL_EVAL_GAMES = 500 # Evaluation games used for the smoke-test baseline and final score

# Experiment 1 settings: history-length sweep
HISTORY_LENGTHS = (1, 2, 3, 4) # (1,2) # History lengths to compare in Experiment 1; please expand this range for your study
HISTORY_SWEEP_EPSILON = 0.2 # Fixed epsilon while comparing history lengths
SWEEP_NUM_EPISODES = 10_000 # Training episodes in each run of Experiments 1 and 2
SWEEP_RUNS = 20 #10 # Default `num_runs` in Experiments 1 and 2 for quick testing
SWEEP_EVAL_GAMES = 500 # Evaluation games used to score each tested setting in Experiments 1 and 2

# Experiment 2 settings: epsilon sweep
EPSILON_SWEEP_HISTORY_LENGTH = 2 #1 # Placeholder history length for Experiment 2; update after Experiment 1
EPSILON_VALUES = (0.0, 0.1, 0.2, 0.3, 0.5) # (0.0, 0.1) # Starting epsilon values for Experiment 2; expand in the interval [0,1)

# Experiment 3 settings: learning curves
CURVE_HISTORY_LENGTH = 2 # Placeholder history length for Experiment 3; update after Experiment 1
CURVE_EPSILON = 0.3 # Placeholder epsilon for Experiment 3; update after Experiment 2
CURVE_NUM_EPISODES = 10_000 # Training episodes in each learning-curve run
CURVE_RUNS = 20 # Default `num_runs` in Experiment 3 for quick testing
CURVE_EVAL_INTERVAL = 500 # Training episodes between two points on the learning curve
CURVE_EVAL_GAMES = 200 # Evaluation games averaged into each point on the learning curve

# Example-call settings
REPORT_SWEEP_RUNS = 10 # Suggested `num_runs` in the final Experiment 1 and 2 calls; you may want this higher than `SWEEP_RUNS`
for steadier reported results
REPORT_CURVE_RUNS = 10 # Suggested `num_runs` in the final Experiment 3 call; you may want this higher than `CURVE_RUNS` for a
smoother final figure

# Baseline settings
BASELINE_EVAL_GAMES = 2_000 # Evaluation games used to estimate the random baseline shown in plots/tables

State = Tuple

# =====
# Q-LEARNING AGENT (YOU IMPLEMENT)
# =====

class QLearningAgent:
    """
    Tabular Q-learning agent.

    Update rule:
        Q(s,a) <- Q(s,a) + eta * [target - Q(s,a)]
    """

    def __init__(
        self,
        env: MastermindEnv,
        eta: float = 0.1,
        gamma: float = 0.99,
        epsilon: float = 0.1,
    ):
        self.env = env
        self.eta = eta
        self.gamma = gamma
        self.epsilon = epsilon

        # QTable is provided separately in qtable.py. You only need to use its
        # public API; you do not need to implement its internal storage.
        self.Q = QTable(num_actions=env.num_codes)
```

```
def select_action(self, state: State, explore: bool = True) -> int:
    """
    Epsilon-greedy action selection.

    Hints:
    - np.random.randint(self.env.num_codes) gives a random action
    - self.Q.get_best_action(state) gives the greedy action
    """
    # -----
    # STUDENT TASK: implement epsilon-greedy action selection.
    # This is a placeholder implementation that always selects a random action.
    # -----

    if explore and np.random.random() < self.epsilon:
        random_action = int(np.random.randint(self.env.num_codes))
        return int(random_action)
    best_action = self.Q.get_best_action(state)
    return best_action
```

```
def update(self, state, action, reward, next_state, done):
    currentQvalue = self.Q.get(state, action)
```

```
    if done:
        futureReward = 0
    else:
        futureReward = self.Q.get_max_value(next_state)
```

```
    target = reward + self.gamma * futureReward
    error = target - currentQvalue
    updatedQValue = currentQvalue + self.eta * error
    self.Q.set(state, action, updatedQValue)
    """
```

Q-learning update.

```
    Hints:
    - self.Q.get(state, action) returns Q-value
    - self.Q.get_max_value(next_state) returns max Q over actions
    - self.Q.set(state, action, value) stores Q-value
    """
```

```
    # -----
    # STUDENT TASK: implement the Q-learning update
    # -----
```

```
pass
```

```
def train_episode(self) -> Tuple[float, bool, int]:
    """Train for one episode. Returns (total_reward, won, num_turns)."""
    state = self.env.reset()
    total_reward = 0.0
    info = {"won": False} # overwritten each step; default is for safety

    while not self.env.done:
        action = self.select_action(state, explore=True)
        next_state, reward, done, info = self.env.step(action)
        total_reward += reward
        self.update(state, action, reward, next_state, done)
        state = next_state

    return total_reward, info["won"], self.env.turn
```

```
# =====
# EVALUATION AND HELPERS (PROVIDED)
# =====
```

```
def evaluate_agent(agent: QLearningAgent, num_episodes: int = 500) -> Tuple[float, float]:
    """
```

Evaluate an agent over many episodes.

```
    Returns:
        tuple: (win_rate, mean_turns)
```

```
    In the average-turns metric, each unsolved evaluation game contributes
    `max_turns + 1`. This is the turns value assigned to failed games in the metric.
    """
```

```
wins = 0
all_turns = []
```

```
for _ in range(num_episodes):
    state = agent.env.reset()
    while not agent.env.done:
        action = agent.select_action(state, explore=False)
        state, _, _, info = agent.env.step(action)
```

```
    if info["won"]:
        wins += 1
        all_turns.append(agent.env.turn)
```

```

        else:
            all_turns.append(agent.env.max_turns + 1)

    return wins / num_episodes, float(np.mean(all_turns))

def train_with_tracking(
    env: MastermindEnv,
    eta: float,
    gamma: float,
    epsilon: float,
    num_episodes: int,
    eval_interval: int = 500,
    eval_games: int = CURVE_EVAL_GAMES,
) -> Tuple[QLearningAgent, dict]:
    """
    Train an agent and record evaluation snapshots.

    Returns:
        tuple: (trained_agent, history_dict)

    The history dict has keys "episodes", "win_rates", and "avg_turns". A new
    entry is recorded every `eval_interval` episodes.
    """
    agent = QLearningAgent(env, eta=eta, gamma=gamma, epsilon=epsilon)
    history = {"episodes": [], "win_rates": [], "avg_turns": []}

    for episode in range(num_episodes):
        agent.train_episode()

        if (episode + 1) % eval_interval == 0:
            win_rate, avg_turns = evaluate_agent(
                agent,
                num_episodes=eval_games,
            )
            history["episodes"].append(episode + 1)
            history["win_rates"].append(win_rate)
            history["avg_turns"].append(avg_turns)

    return agent, history

def compute_random_baseline(
    num_games: int = BASELINE_EVAL_GAMES,
) -> Tuple[float, float]:
    """
    Evaluate a uniformly random policy.

    Returns:
        tuple: (win_rate, mean_turns)

    The average-turns metric matches `evaluate_agent`: each unsolved game
    contributes `max_turns + 1`.
    The random baseline does not depend on `history_length` because it ignores
    the state and samples actions uniformly.
    """
    env = MastermindEnv(
        num_positions=NUM_POSITIONS,
        num_colors=NUM_COLORS,
        max_turns=MAX_TURNS,
    )

    wins = 0
    all_turns = []

    for _ in range(num_games):
        env.reset()
        while not env.done:
            env.step(int(np.random.randint(env.num_codes)))

        if env.history[-1][1] == env.num_positions:
            wins += 1
            all_turns.append(env.turn)
        else:
            all_turns.append(env.max_turns + 1)

    return wins / num_games, float(np.mean(all_turns))

def print_summary_table(title, label, results):
    """
    Print a compact results table.

    Args:
        title: Table heading.
        label: Column header for the tested setting.
        results: Mapping from setting value to a dict containing means/stds.
    """

```

```

"""
print("\n" + "-" * 56)
print(title)
print("-" * 56)
print(f"{label:<12} {'Win Rate':<20} {'Avg Turns':<20}")
print("-" * 56)

for setting in sorted(results):
    r = results[setting]
    print(
        f"{str(setting):<12}"
        f"{r['win_rate_mean'] * 100:>5.1f}% +/- {r['win_rate_std'] * 100:<5.1f}%  "
        f"{r['turns_mean']:>5.2f} +/- {r['turns_std']:<5.2f}"
    )

# =====
# PLOTTING TIPS (for your reference)
# =====

# The experiments below ask you to create plots. These are the main matplotlib
# calls you will likely need. Add your plotting code in the marked STUDENT TASK
# sections below:
#
# fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
# ax1.plot(x, means, "o-")
# ax1.fill_between(x, means - stds, means + stds, alpha=0.2)
# ax1.axhline(y=baseline_value, color="red", linestyle="--", label="Random")
# ax1.set_xlabel("...")
# ax1.set_ylabel("...")
# ax1.set_title("...")
# ax1.legend()
# ax1.grid(True, alpha=0.3)
# fig.tight_layout()
# fig.savefig("my_plot.png", dpi=150)
# plt.show()

# =====
# IMPLEMENTATION CHECK (run before the experiments)
# =====

def test_implementation():
    """
    Quick debugging check to verify your implementation works.
    Run this before the other experiments.

    This is not your final experiment configuration and is not intended to be
    used as evidence in your report.
    """
    print("-" * 60)
    print("TESTING YOUR IMPLEMENTATION")
    print("-" * 60)
    print("This is a debugging check only, not a final experiment setup.")

    env = MastermindEnv(
        num_positions=NUM_POSITIONS,
        num_colors=NUM_COLORS,
        max_turns=MAX_TURNS,
        history_length=SMOKE_TEST_HISTORY_LENGTH,
    )

    print(f"\nGame: {env.num_positions} positions x {env.num_colors} colors = {env.num_codes} codes")

    baseline_wr, baseline_turns = compute_random_baseline(
        num_games=SMOKE_TEST_FINAL_EVAL_GAMES,
    )
    print(
        f"Random baseline: {baseline_wr:.1%} win rate, Avg Turns = {baseline_turns:.2f}"
    )

    print(f"\nTraining your agent ({SMOKE_TEST_NUM_EPISODES} episodes)...")
    agent, history = train_with_tracking(
        env,
        eta=ETA,
        gamma=GAMMA,
        epsilon=SMOKE_TEST_EPSILON,
        num_episodes=SMOKE_TEST_NUM_EPISODES,
        eval_interval=SMOKE_TEST_EVAL_INTERVAL,
        eval_games=SMOKE_TEST_CURVE_EVAL_GAMES,
    )

    for episode, win_rate, avg_turns in zip(
        history["episodes"],
        history["win_rates"],
        history["avg_turns"],

```

```

):
    print(f" Episode {episode}: Win Rate = {win_rate:.1%}, Avg Turns = {avg_turns:.2f}")

final_win_rate, final_turns = evaluate_agent(
    agent,
    num_episodes=SMOKE_TEST_FINAL_EVAL_GAMES,
)
print(f"\nFinal: Win Rate = {final_win_rate:.1%}, Avg Turns = {final_turns:.2f}")

if final_win_rate > 0.8:
    print("\nImplementation sanity check passed!")
else:
    print("\nWin rate is still near the random baseline; check your code.")

return agent

```

```

# =====
# EXPERIMENT 1: HISTORY LENGTH
# =====

```

```

def experiment_history_length(
    history_lengths: Tuple[int, ...] = HISTORY_LENGTHS,
    epsilon: float = HISTORY_SWEEP_EPSILON,
    eval_games: int = SWEEP_EVAL_GAMES,
    num_runs: int = SWEEP_RUNS,
    num_episodes: int = SWEEP_NUM_EPISODES,
):
    """
    Investigate history length.

    This function prints a summary table and prepares the data for a
    comparison plot.

    The default values are a quick testing setup only, not final report
    settings. Update `history_lengths`, `epsilon`, or `eval_games` if you want
    to test a different setup, and pass a more substantial `num_runs`
    explicitly for your actual results. In this experiment, keep `epsilon`
    fixed while comparing different history lengths, and keep the learning
    rate (`eta`) and discount factor (`gamma`) fixed at the provided values.
    `eval_games` is the number of evaluation games used to estimate the final
    win rate and average turns for each setting.
    """
    print("=" * 60)
    print("EXPERIMENT: HISTORY LENGTH")
    print("=" * 60)

    results = {}

    for history_length in history_lengths:
        print(f"\nHistory length = {history_length}:")
        win_rates = []
        avg_turns = []

        for run in range(num_runs):
            env = MastermindEnv(
                num_positions=NUM_POSITIONS,
                num_colors=NUM_COLORS,
                max_turns=MAX_TURNS,
                history_length=history_length,
            )
            agent = QLearningAgent(env, eta=ETA, gamma=GAMMA, epsilon=epsilon)
            for _ in range(num_episodes):
                agent.train_episode()
            wr, turns = evaluate_agent(agent, num_episodes=eval_games)
            win_rates.append(wr)
            avg_turns.append(turns)
            print(f" Run {run + 1}: Win Rate = {wr:.1%}, Turns = {turns:.2f}")

        results[history_length] = {
            "win_rate_mean": float(np.mean(win_rates)),
            "win_rate_std": float(np.std(win_rates)),
            "turns_mean": float(np.mean(avg_turns)),
            "turns_std": float(np.std(avg_turns)),
        }

    baseline_wr, baseline_turns = compute_random_baseline()
    print_summary_table("History Length Summary", "history", results)

    x = np.array(history_lengths, dtype=float)
    win_means = np.array([results[h]["win_rate_mean"] for h in history_lengths])
    win_stds = np.array([results[h]["win_rate_std"] for h in history_lengths])
    turn_means = np.array([results[h]["turns_mean"] for h in history_lengths])
    turn_stds = np.array([results[h]["turns_std"] for h in history_lengths])

    # -----

```

```
# STUDENT TASK: create your comparison plot here.
# Use `x`, `win_means`, `win_stds`, `turn_means`, `turn_stds`,
# `baseline_wr`, and `baseline_turns`.
# -----
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.errorbar(x, win_means * 100, yerr=win_stds * 100, fmt="o-", capsize=4)
ax1.axhline(baseline_wr * 100, color="red", linestyle="--", label="Random baseline")
ax1.set_xlabel("History Length"); ax1.set_ylabel("Win Rate (%)")
ax1.set_title("Win Rate vs History Length"); ax1.legend(); ax1.grid(True, alpha=0.3)
ax2.errorbar(x, turn_means, yerr=turn_stds, fmt="o-", capsize=4)
ax2.axhline(baseline_turns, color="red", linestyle="--", label="Random baseline")
ax2.set_xlabel("History Length"); ax2.set_ylabel("Avg Turns")
ax2.set_title("Avg Turns vs History Length"); ax2.legend(); ax2.grid(True, alpha=0.3)
fig.tight_layout(); fig.savefig("history_length.png", dpi=150); plt.show()
```

```
return results
```

```
# =====
# EXPERIMENT 2: EXPLORATION RATE
# =====
```

```
def experiment_epsilon(
    history_length: int = EPSILON_SWEEP_HISTORY_LENGTH,
    epsilon_values: Tuple[float, ...] = EPSILON_VALUES,
    eval_games: int = SWEEP_EVAL_GAMES,
    num_runs: int = SWEEP_RUNS,
    num_episodes: int = SWEEP_NUM_EPISODES,
):
    """
    Investigate exploration rate (`epsilon`) using your chosen history length.

    This function prints a summary table and prepares the data for a
    comparison plot.

    The default `history_length` is a placeholder for a first run only; update
    it based on your results from Experiment 1. The default `epsilon_values`
    are only a quick starting point; expand this range for your actual study.
    Start with a coarse sweep using a small number of well-spaced epsilon
    values, and only increase the resolution if needed. You can also change
    `eval_games` if needed. The default `num_runs` is also only for quick
    testing, so pass a more substantial value explicitly for your actual
    results. `eval_games` is the number of evaluation games used to estimate
    the final win rate and average turns for each epsilon. Keep the learning
    rate (`eta`) and discount factor (`gamma`) fixed at the provided values.
    """
    print("=" * 60)
    print("EXPERIMENT: EXPLORATION RATE (EPSILON)")
    print("=" * 60)

    results = {}

    for epsilon in epsilon_values:
        print(f"\nEpsilon = {epsilon}:")
        win_rates = []
        avg_turns = []

        for run in range(num_runs):
            env = MastermindEnv(
                num_positions=NUM_POSITIONS,
                num_colors=NUM_COLORS,
                max_turns=MAX_TURNS,
                history_length=history_length,
            )
            agent = QLearningAgent(env, eta=ETA, gamma=GAMMA, epsilon=epsilon)
            for _ in range(num_episodes):
                agent.train_episode()
            wr, turns = evaluate_agent(agent, num_episodes=eval_games)
            win_rates.append(wr)
            avg_turns.append(turns)
            print(f"  Run {run + 1}: Win Rate = {wr:.1%}, Turns = {turns:.2f}")

        results[epsilon] = {
            "win_rate_mean": float(np.mean(win_rates)),
            "win_rate_std": float(np.std(win_rates)),
            "turns_mean": float(np.mean(avg_turns)),
            "turns_std": float(np.std(avg_turns)),
        }

    baseline_wr, baseline_turns = compute_random_baseline()
    print_summary_table("Epsilon Summary", "epsilon", results)

    x = np.array(epsilon_values, dtype=float)
    win_means = np.array([results[e]["win_rate_mean"] for e in epsilon_values])
    win_stds = np.array([results[e]["win_rate_std"] for e in epsilon_values])
```

```

turn_means = np.array([results[e]["turns_mean"] for e in epsilon_values])
turn_std = np.array([results[e]["turns_std"] for e in epsilon_values])

# -----
# STUDENT TASK: create your comparison plot here.
# Use `x`, `win_means`, `win_std`, `turn_means`, `turn_std`,
# `baseline_wr`, and `baseline_turns`.
# -----

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

ax1.errorbar(x, win_means * 100, yerr=win_std * 100, fmt="o-", capsize=4, label="Q-learning")
ax1.axhline(baseline_wr * 100, color="red", linestyle="--", label="Random baseline")
ax1.set_xlabel("Epsilon")
ax1.set_ylabel("Win Rate (%)")
ax1.set_title("Win Rate vs Epsilon")
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.errorbar(x, turn_means, yerr=turn_std, fmt="o-", capsize=4, label="Q-learning")
ax2.axhline(baseline_turns, color="red", linestyle="--", label="Random baseline")
ax2.set_xlabel("Epsilon")
ax2.set_ylabel("Avg Turns")
ax2.set_title("Avg Turns vs Epsilon")
ax2.legend()
ax2.grid(True, alpha=0.3)

fig.tight_layout()
fig.savefig("epsilon.png", dpi=150)
plt.show()

return results

```

```

# =====
# EXPERIMENT 3: LEARNING CURVES
# =====

```

```

def experiment_learning_curves(
    history_length: int = CURVE_HISTORY_LENGTH,
    epsilon: float = CURVE_EPSILON,
    eval_games: int = CURVE_EVAL_GAMES,
    eval_interval: int = CURVE_EVAL_INTERVAL,
    num_runs: int = CURVE_RUNS,
    num_episodes: int = CURVE_NUM_EPISODES,
):
    """
    Prepare the learning-curve data needed to plot error bands.

    Run multiple independent training runs and plot:
    - win rate over training (mean +/- std)
    - average turns over training (mean +/- std)

    This function prepares the learning-curve data needed for your report
    plot.

    Use your chosen history length and epsilon from the previous experiments.
    The default values here are placeholders for a first run only; update them
    based on your results from Experiments 1 and 2. You can also change
    `eval_interval` to control how often the learning curve is measured.
    Smaller `eval_interval` gives more points on the curve but makes training
    slower because evaluation is performed more often. `eval_games` controls
    how many evaluation games are averaged into each point on the curve. The
    default `num_runs` is only for quick testing, so pass a more substantial
    value explicitly for your actual results. Keep the learning rate (`eta`)
    and discount factor (`gamma`) fixed at the provided values.
    """
    print("=" * 60)
    print("EXPERIMENT: LEARNING CURVES")
    print("=" * 60)

    all_win_rates = []
    all_avg_turns = []
    episodes = None

    for run in range(num_runs):
        print(f"Run {run + 1}/{num_runs}")
        env = MastermindEnv(
            num_positions=NUM_POSITIONS,
            num_colors=NUM_COLORS,
            max_turns=MAX_TURNS,
            history_length=history_length,
        )
        _, history = train_with_tracking(
            env,
            eta=ETA,

```



```

        gamma=GAMMA,
        epsilon=epsilon,
        num_episodes=num_episodes,
        eval_interval=eval_interval,
        eval_games=eval_games,
    )
    all_win_rates.append(history["win_rates"])
    all_avg_turns.append(history["avg_turns"])
    if episodes is None:
        episodes = history["episodes"]

win_rates_array = np.array(all_win_rates)
avg_turns_array = np.array(all_avg_turns)

win_rate_mean = np.mean(win_rates_array, axis=0)
win_rate_std = np.std(win_rates_array, axis=0)
turns_mean = np.mean(avg_turns_array, axis=0)
turns_std = np.std(avg_turns_array, axis=0)

baseline_wr, baseline_turns = compute_random_baseline()

# -----
# STUDENT TASK: create your learning-curve plot here.
# Use `episodes`, `win_rate_mean`, `win_rate_std`, `turns_mean`,
# `turns_std`, `baseline_wr`, and `baseline_turns`.
# -----

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(episodes, win_rate_mean * 100, "b-")
ax1.fill_between(episodes, (win_rate_mean - win_rate_std) * 100, (win_rate_mean + win_rate_std) * 100, alpha=0.2)
ax1.axhline(baseline_wr * 100, color="red", linestyle="--", label="Random baseline")
ax1.set_xlabel("Episodes"); ax1.set_ylabel("Win Rate (%)"); ax1.set_title("Learning Curve: Win Rate")
ax1.legend(); ax1.grid(True, alpha=0.3)
ax2.plot(episodes, turns_mean, "b-")
ax2.fill_between(episodes, turns_mean - turns_std, turns_mean + turns_std, alpha=0.2)
ax2.axhline(baseline_turns, color="red", linestyle="--", label="Random baseline")
ax2.set_xlabel("Episodes"); ax2.set_ylabel("Avg Turns"); ax2.set_title("Learning Curve: Avg Turns")
ax2.legend(); ax2.grid(True, alpha=0.3)
fig.tight_layout(); fig.savefig("learning_curves.png", dpi=150); plt.show()

```

```

# =====
# ANALYSIS QUESTIONS (Answer in your report)
# =====

```

```

"""
ANALYSIS QUESTIONS
=====

```

Implementation is assessed implicitly through the correctness and quality of the experiments and analysis below, rather than as a separate standalone code mark. Please state the configuration you used in your report, especially `NUM_COLORS`, the parameter values tested in each experiment, and the final values used for your reported plots and tables. Compare results to a random baseline computed under the same configuration.

Q1: IMPLEMENTATION

Explain the Q-learning update rule in your own words:

- What is the "target" and why do we compute it differently for terminal vs non-terminal states?
- What role does the learning rate (`eta`) play?
- What role does the discount factor (`gamma`) play?

Q2: HISTORY LENGTH RESULTS

Please include your results table and plot, and compare your results against a random baseline.

- What win rate do you observe with `history_length = 1`?
Compare this to the random baseline. Explain why the agent cannot achieve 100% performance with only the most recent feedback.
- What is the optimal history length in your study?
Explain why it gives the best overall performance. Is the concept of markovianity relevant here?
- As `history_length` increases, how does the size of the state space change? Based on your results and this observation, discuss the trade-off between having a richer state representation and the number of training episodes needed to learn a good policy. Did you observe any evidence of this trade-off in your experiments?
(Hint: how does history length affect the size of the state space?)

Q3: EXPLORATION RESULTS

Please include your results table and plot, and compare your results against a random baseline.

- (a) What win rate do you observe with ``epsilon = 0``?
In many problems this is bad for exploration, but in this setup you may still observe good performance. Compare your result to the random baseline and to your positive-epsilon results. Explain why this may happen, relating your answer to the initial Q-values, the reward structure, and the fact that greedy tie-breaking is random when actions have equal Q-values.
- (b) What epsilon value or regime gives the best final performance? Is there a clear winner, or do multiple values work similarly well? If final performance is similar across several values, discuss whether you observe any differences in convergence speed or stability during training.
- (c) Explain the exploration-exploitation trade-off:
- What happens if epsilon is too low?
- What happens if epsilon is too high?
- Why do we set ``explore=False`` during evaluation?

Q4: LEARNING DYNAMICS

Please include your learning curve plot (with error bands from multiple runs). When comparing parameter settings, consider both the mean performance and the variability across runs; if several settings have similar means, discuss whether one appears more stable.

- (a) Approximately how many episodes are needed to reach good performance? Compare the agent's performance to the random baseline. Does the agent continue improving after that, or does it plateau?
- (b) Why is it important to run multiple independent training runs and report mean +/- std, rather than just showing a single run? Approximately how many repetitions were needed before your qualitative conclusions looked stable?
- (c) In the average-turns metric, each unsolved game contributes ``max_turns + 1`` turns. Why is this better than only averaging turns for games the agent won?

PASSING THE ASSIGNMENT

This assignment uses a threshold-style assessment approach. To achieve a pass on the assignment, your submission should include functional code that shows evidence of learning beyond a random baseline, together with correct responses to about half of the assessed written content.

It is strongly recommended that you attempt all 4 questions.

Please see ``README.md`` for the full assessment guidance and the brief mapping to the module learning objectives.

"""

```
if __name__ == "__main__":
    agent = test_implementation()

    # Uncomment these once your implementation is working.
    # These are the actual experiment calls that produce the tables and
    # computed results for your report. The function defaults are only for
    # quick tests, so the separate `REPORT_*` values below can be set higher
    # for steadier final figures. Add your own plotting code in the marked
    # STUDENT TASK sections above.
    history_results = experiment_history_length(
        history_lengths=HISTORY_LENGTHS,      # <-- update this to test a different range
        epsilon=HISTORY_SWEEP_EPSILON,        # <-- keep this fixed during Experiment 1
        eval_games=SWEEP_EVAL_GAMES,
        num_runs=REPORT_SWEEP_RUNS,
        num_episodes=SWEEP_NUM_EPISODES,
    )
    epsilon_results = experiment_epsilon(
        history_length=EPSILON_SWEEP_HISTORY_LENGTH, # <-- update this based on your Experiment 1 results
        epsilon_values=EPSILON_VALUES,              # <-- expand this range for your actual study
        eval_games=SWEEP_EVAL_GAMES,
        num_runs=REPORT_SWEEP_RUNS,
        num_episodes=SWEEP_NUM_EPISODES,
    )
    # experiment_learning_curves(
    #     history_length=CURVE_HISTORY_LENGTH, # <-- update this based on your Experiment 1 results
    #     epsilon=CURVE_EPSILON,              # <-- update this based on your Experiment 2 results
    #     eval_games=CURVE_EVAL_GAMES,
    #     eval_interval=CURVE_EVAL_INTERVAL,
    #     num_runs=REPORT_CURVE_RUNS,
```

